DSA MINI PROJECT

SRN	NAME
PES2UG24CS910	PRAJWAL.M
PES2UG24CS150	DEEPTHI.V
PES2UG24CS917	R.P PRANAV
PES2UG24CS159	DIMPAL.N

Q1: Randomised Algorithms

CODE :

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

typedef struct TreapNode {
  int key, priority;
  struct TreapNode *left, *right;
} TreapNode;

TreapNode *newNode(int key) {
  TreapNode *node = (TreapNode *)malloc(sizeof(TreapNode));
  if (!node) {
    fprintf(stderr, "Memory allocation failed\n");
    exit(EXIT_FAILURE);
  }
  node->key = key;
  node->priority = rand() % 100;
  node->left = NULL;
  node->right = NULL;
  return node;
}

TreapNode *rightRotate(TreapNode *y) {
  TreapNode *x = y->left;
  TreapNode *T2 = x->right;

  x->right = y;
  y->left = T2;

  return x; /* new root */
}

TreapNode *leftRotate(TreapNode *x) {
  TreapNode *y = x->right;
  TreapNode *T2 = y->left;

  y->left = x;
  x->right = T2;

  return y; /* new root */
}
```

```c
TreapNode *search(TreapNode *root, int key) {
  if (root == NULL || root->key == key)
    return root;

  if (key > root->key)
    return search(root->right, key);

  return search(root->left, key);
}

TreapNode *insert(TreapNode *root, int key) {
  if (root == NULL)
    return newNode(key);

  if (key <= root->key) {
    root->left = insert(root->left, key);

    /* Heap property violated → right-rotate to push new node up */
    if (root->left->priority > root->priority)
      root = rightRotate(root);
  } else {
    root->right = insert(root->right, key);

    /* Heap property violated → left-rotate to push new node up */
    if (root->right->priority > root->priority)
      root = leftRotate(root);
  }

  return root;
}

/* ── Delete ──────────────────────────────────────────────────────────── */

/*
 * Three cases once we locate the node:
 *   1. Leaf          → free it directly.
 *   2. One child     → replace node with that child.
 *   3. Two children  → rotate the higher-priority child up, then recurse.
 *       This keeps pushing the target node downward until it becomes case 1/2.
```

```c
TreapNode *deleteNode(TreapNode *root, int key) {
  if (root == NULL)
    return NULL;

  /* ── Navigate to the node ── */
  if (key < root->key) {
    root->left = deleteNode(root->left, key);
    return root;
  }
  if (key > root->key) {
    root->right = deleteNode(root->right, key);
    return root;
  }

  /* ── Key found at root ── */

  /* Case 1 & 2a: left child is absent */
  if (root->left == NULL) {
    TreapNode *temp = root->right;
    free(root);
    return temp;
  }

  /* Case 2b: right child is absent */
  if (root->right == NULL) {
    TreapNode *temp = root->left;
    free(root);
    return temp;
  }

  /* Case 3: both children present — rotate higher-priority child up */
  if (root->right->priority > root->left->priority) {
    root = leftRotate(root);
    root->left = deleteNode(root->left, key);
  } else {
    root = rightRotate(root);
    root->right = deleteNode(root->right, key);
  }
```

```c
  return root;
}

/* ── Inorder Traversal (prints sorted keys with priorities) ──────────── */

void inorder(TreapNode *root) {
  if (root == NULL)
    return;

  inorder(root->left);

  printf("key: %2d | priority: %2d", root->key, root->priority);
  if (root->left)
    printf(" | left: %d", root->left->key);
  if (root->right)
    printf(" | right: %d", root->right->key);
  printf("\n");

  inorder(root->right);
}

/* ── Free the entire tree ──────────────────────────────────────────── */

void freeTree(TreapNode *root) {
  if (root == NULL)
    return;
  freeTree(root->left);
  freeTree(root->right);
  free(root);
}
```

```c
/* ── Driver ──────────────────────────────────────────────────────────── */

void printMenu(void) {
  printf("\n╔═══════════════════════════╗\n");
  printf("╠        TREAP MENU        ╣\n");
  printf("╠═══════════════════════════╣\n");
  printf("╠  1. Insert                ╣\n");
  printf("╠  2. Delete                ╣\n");
  printf("╠  3. Search                ╣\n");
  printf("╠  4. Inorder Traversal     ╣\n");
  printf("╠  5. Clear Tree            ╣\n");
  printf("╠  0. Exit                  ╣\n");
  printf("╚═══════════════════════════╝\n");
  printf("Choice: ");
}

int main(void) {
  srand((unsigned)time(NULL));

  TreapNode *root = NULL;
  int choice, key;

  printf("=== Treap — Interactive Demo ===\n");

  do {
    printMenu();

    if (scanf("%d", &choice) != 1) {
      /* flush bad input */
      while (getchar() != '\n')
        ;
      choice = -1;
    }

    switch (choice) {

    case 1:
      printf("Enter key to insert: ");
      scanf("%d", &key);
      root = insert(root, key);
      printf("Inserted %d into the treap.\n", key);
      break;

    case 2:
      if (root == NULL) {
        printf("Tree is empty. Nothing to delete.\n");
        break;
      }
      printf("Enter key to delete: ");
      scanf("%d", &key);
      if (search(root, key)) {
        root = deleteNode(root, key);
        printf("Deleted %d from the treap.\n", key);
      } else {
        printf("Key %d not found in the treap.\n", key);
      }
      break;
```

```c
    case 3:
      if (root == NULL) {
        printf("Tree is empty.\n");
        break;
      }
      printf("Enter key to search: ");
      scanf("%d", &key);
      printf("Search(%d): %s\n", key,
             search(root, key) ? "Found ✓" : "Not Found ✗");
      break;

    case 4:
      if (root == NULL) {
        printf("Tree is empty.\n");
      } else {
        printf("\n--- Inorder Traversal ---\n");
        inorder(root);
      }
      break;

    case 5:
      freeTree(root);
      root = NULL;
      printf("Tree cleared.\n");
      break;

    case 0:
      printf("Exiting. Freeing memory...\n");
      freeTree(root);
      break;

    default:
      printf("Invalid choice. Please enter 0-5.\n");
      break;
    }

  } while (choice != 0);

  return 0;
}
```

OUTPUT:

```
Last login: Wed Apr 15 07:51:15 on console
/Users/prajwalm/Programming/C\ programs/project/treap ; exit;
prajwalm@Prajwals-MacBook-Air ~ % /Users/prajwalm/Programming/C\ programs/project/treap ; exit;
=== Treap — Interactive Demo ===

╔═══════════════════╗
║        TREAP MENU        ╕
╞═══════════════════╡
  1. Insert
  2. Delete
  3. Search
  4. Inorder Traversal
  5. Clear Tree
  0. Exit
╚═══════════════════╝
Choice: 4
Tree is empty.

╔═══════════════════╗
║        TREAP MENU        ╕
╞═══════════════════╡
  1. Insert
  2. Delete
  3. Search
  4. Inorder Traversal
  5. Clear Tree
  0. Exit
╚═══════════════════╝
Choice: 1
Enter key to insert: 54
Inserted 54 into the treap.

╔═══════════════════╗
║        TREAP MENU        ╕
╞═══════════════════╡
  1. Insert
  2. Delete
  3. Search
  4. Inorder Traversal
  5. Clear Tree
  0. Exit
╚═══════════════════╝
Choice: 4

--- Inorder Traversal ---
key: 54 | priority: 13

╔═══════════════════╗
║        TREAP MENU        ╕
╞═══════════════════╡
  1. Insert
  2. Delete
  3. Search
  4. Inorder Traversal
  5. Clear Tree
  0. Exit
╚═══════════════════╝
Choice: 1
Enter key to insert: 23
Inserted 23 into the treap.
```

```
╔═══════════╗
╠═══ TREAP MENU ═══╣
╚═══════════╝
  1. Insert
  2. Delete
  3. Search
  4. Inorder Traversal
  5. Clear Tree
  0. Exit
Choice: 1
Enter key to insert: 34
Inserted 34 into the treap.

╔═══════════╗
╠═══ TREAP MENU ═══╣
╚═══════════╝
  1. Insert
  2. Delete
  3. Search
  4. Inorder Traversal
  5. Clear Tree
  0. Exit
Choice: 1
Enter key to insert: 68
Inserted 68 into the treap.

╔═══════════╗
╠═══ TREAP MENU ═══╣
╚═══════════╝
  1. Insert
  2. Delete
  3. Search
  4. Inorder Traversal
  5. Clear Tree
  0. Exit
Choice: 4

─── Inorder Traversal ───
key: 23 | priority: 30
key: 34 | priority: 60 | left: 23 | right: 68
key: 54 | priority: 13
key: 68 | priority: 27 | left: 54

╔═══════════╗
╠═══ TREAP MENU ═══╣
╚═══════════╝
  1. Insert
  2. Delete
  3. Search
  4. Inorder Traversal
  5. Clear Tree
  0. Exit
Choice: 3
Enter key to search: 23
Search(23): Found ✓
```

```
╔═══════════╗
╠═══ TREAP MENU ═══╣
╚═══════════╝
  1. Insert
  2. Delete
  3. Search
  4. Inorder Traversal
  5. Clear Tree
  0. Exit
Choice: 2
Enter key to delete: 68
Deleted 68 from the treap.

╔═══════════╗
╠═══ TREAP MENU ═══╣
╚═══════════╝
  1. Insert
  2. Delete
  3. Search
  4. Inorder Traversal
  5. Clear Tree
  0. Exit
Choice: 4

─── Inorder Traversal ───
key: 23 | priority: 30
key: 34 | priority: 60 | left: 23 | right: 54
key: 54 | priority: 13

╔═══════════╗
╠═══ TREAP MENU ═══╣
╚═══════════╝
  1. Insert
  2. Delete
  3. Search
  4. Inorder Traversal
  5. Clear Tree
  0. Exit
Choice: 5
Tree cleared.

╔═══════════╗
╠═══ TREAP MENU ═══╣
╚═══════════╝
  1. Insert
  2. Delete
  3. Search
  4. Inorder Traversal
  5. Clear Tree
  0. Exit
Choice: 4
Tree is empty.

╔═══════════╗
╠═══ TREAP MENU ═══╣
```

TASKS :

1. Prove that the expected number of comparisons in Randomised Quicksort is O(n log n)

Consider an indicator random variable X_{ij}, which is 1 if elements $i$and $j$are compared, otherwise 0.
Two elements are compared only if one of them is chosen as the first pivot among those elements.

The probability of this is:

$$P(X_{ij} = 1) = \frac{2}{j - i + 1}$$

Using linearity of expectation:

$$E[X] = \sum_{i<j} P(X_{ij}) = \sum_{i<j} \frac{2}{j - i + 1}$$

This summation evaluates to $O(n\log\ n)$.
Hence, the expected number of comparisons in Randomised Quicksort is $O(n\log\ n)$.

---

2. Show that no deterministic comparison-based sorting algorithm can avoid O(n²) worst-case behaviour when an adversary sees the pivot-selection strategy

In deterministic Quicksort, the pivot is chosen using a fixed rule.
An adversary can construct the input such that the pivot always results in the most unbalanced partition.

This leads to the recurrence:

$$T(n) = T(n - 1) + O(n)$$

Solving this:

$$T(n) = O(n^2)$$

Thus, deterministic comparison-based sorting cannot avoid worst-case $O(n^2)$behaviour.

---

3. Define a Treap. Prove that a Treap built by inserting n keys with random priorities has expected height O(log n)

A Treap is a binary search tree that also satisfies the heap property based on randomly assigned priorities.

- BST property: left subtree keys < root < right subtree keys
- Heap property: parent priority > child priorities

Since priorities are random, the Treap structure is equivalent to a randomly built BST.
The expected height of a random BST is $O(\log\ n)$.

Therefore, the expected height of a Treap is $O(\log\ n)$.

---

4. Implement Treap insert, delete, and search operations. Derive the expected time complexity of each

In a Treap:

- Search is performed like in a BST
- Insert is done using BST insertion followed by rotations to maintain heap property
- Delete uses rotations to move the node down until it can be removed

All operations depend on the height of the tree.

Since the expected height is $O(\log\ n)$:

- Search: $O(\log\ n)$
- Insert: $O(\log\ n)$
- Delete: $O(\log\ n)$

---

5. Compare Treaps with AVL trees and Red-Black trees. When does the constant-factor simplicity of Treaps outweigh the guaranteed O(log n) height of balanced BSTs

Treaps maintain balance using random priorities, while AVL and Red-Black trees enforce strict balancing rules.

- AVL trees are strictly balanced and may require frequent rotations
- Red-Black trees are less strict but still guarantee balance
- Treaps are simpler to implement and require fewer cases

Treaps are preferred when simplicity and good average-case performance are sufficient.
AVL and Red-Black trees are preferred when guaranteed worst-case performance is required.

Q2: How Uber Imagines Infrastructures

1.Design an efficient algorithm to preprocess the tree and answer subtree prime-count queries efficiently for multiple queries.

We use DFS traversal to preprocess the tree. For each node, we traverse its entire subtree and count how many nodes have prime values. We store this count in ans[u] so any query is just a direct lookup — O(1) per query after preprocessing.

2.Implement the function efficiencyScores(...) that returns the number of prime-valued nodes in each queried subtree.

efficiencyScores function

```
int efficiencyScores(int u, int parent){
   int cnt = 0;

   if(prime[val[u]])
      cnt = 1;

   for(int i = head[u]; i != -1; i = next[i]){
      int v = to[i];
      if(v != parent)
         cnt += efficiencyScores(v, u);
   }
```

```
    ans[u] = cnt;

    return cnt;

}
```

3.Propose and implement an efficient method to determine whether efficiency scores are prime for all nodes (consider constraints up to 10^5).

Prime checking method

We use Sieve of Eratosthenes. We make an array `prime[]` of size equal to max efficiency score. We mark all numbers as prime initially, then for each number starting from 2, we mark all its multiples as not prime. This way checking if any value is prime is O(1) after O(n log log n) preprocessing.

4.Analyze the time complexity of:
○Prime preprocessing
○Tree traversal and preprocessing
○Each query

- Prime preprocessing: O(n log log n) — Sieve of Eratosthenes
- Tree traversal: O(n) — each node visited exactly once in DFS
- Each query: O(1) — direct lookup in ans[] array

5.Discuss how your approach performs for large inputs (up to 10^5 nodes and queries) and suggest optimizations if queries were to be handled online

For 10^5 nodes our approach works fine since preprocessing is O(n) and queries are O(1). For online queries (where tree can change between queries) this approach won't work directly since we'd have to rerun DFS every time. One optimization would be using Euler tour + prefix sums so subtree queries can still be answered in O(1) even if values update, without redoing full DFS.

Code:

```
#include <stdio.h>
#include <stdbool.h>

#define MAX 100005

int head[MAX], to[2*MAX], next[2*MAX], edge=0;
int val[MAX], prime[MAX], ans[MAX];

void add(int u,int v){
    to[edge]=v;
    next[edge]=head[u];
    head[u]=edge++;

    to[edge]=u;
    next[edge]=head[v];
    head[v]=edge++;
}

void makePrime(int n){
    for(int i=0;i<=n;i++)
        prime[i]=1;

    prime[0]=prime[1]=0;

    for(int i=2;i*i<=n;i++){
        if(prime[i]){
            for(int j=i*i;j<=n;j+=i)
                prime[j]=0;
        }
    }
}

int solve(int u,int parent){
    int cnt=0;

    if(prime[val[u]])
        cnt=1;
```

```c
    for(int i=head[u];i!=-1;i=next[i]){
        int v=to[i];
        if(v!=parent)
            cnt+=solve(v,u);
    }

    ans[u]=cnt;
    return cnt;
}

int main(){
    int n;
    printf("Enter number of nodes: ");
    scanf("%d",&n);

    for(int i=1;i<=n;i++)
        head[i]=-1;

    int maxv=0;

    for(int i=1;i<=n;i++){
        printf("Enter value of node %d: ",i);
        scanf("%d",&val[i]);
        if(val[i]>maxv)
            maxv=val[i];
    }

    for(int i=0;i<n-1;i++){
        int u,v;
        printf("Enter edge %d: ",i+1);
        scanf("%d %d",&u,&v);
        add(u,v);
    }

    makePrime(maxv);
    solve(1,-1);

    int q;
    printf("Enter number of queries: ");
    scanf("%d",&q);

    printf("\n=== Query Results ===\n");
    while(q--){
        int x;
        printf("Enter node: ");
        scanf("%d",&x);
        printf("Answer: %d\n",ans[x]);
    }

    return 0;
}
```

Output:

```
pranav@Rs-MacBook-Air-2 DAA % clang -g subtree_prime_count.c -o subtree_prime_count
./subtree_prime_count
Enter number of nodes: 5
Enter value of node 1: 1
Enter value of node 2: 7
Enter value of node 3: 89
Enter value of node 4: 45
Enter value of node 5: 37
Enter edge 1: 1 2
Enter edge 2: 1 3
Enter edge 3: 2 4
Enter edge 4: 2 5
Enter number of queries: 3

=== Query Results ===
Enter node: 2
Answer: 2
Enter node: 1
Answer: 3
Enter node: 4
Answer: 0
pranav@Rs-MacBook-Air-2 DAA %
```

Subtree used:

```
      1
    /   \
   2     3
 /    \
4      5
```

Q3: N-Queens Secure Vault Placement

Tasks

1. Design a backtracking algorithm to solve the N-Queens problem with blocked cells, ensuring all constraints are satisfied.

```
Initialize col[0..N-1] = 0
Initialize diag1[0..2N-1] = 0
Initialize diag2[0..2N-1] = 0

Call NQueensBlocked(A, 0)

NQueensBlocked(A[0..N-1][0..N-1], row)

if row == N
   print solution / count++
   return

for c = 0 to N-1
   if A[row][c] == -1
      continue
```

```
if col[c] == 1 or diag1[row + c] == 1 or diag2[row - c + N - 1] == 1
    continue

A[row][c] = 1
col[c] = 1
diag1[row + c] = 1
diag2[row - c + N - 1] = 1

NQueensBlocked(A, row + 1)

A[row][c] = 0
col[c] = 0
diag1[row + c] = 0
diag2[row - c + N - 1] = 0
```

---

2. Implement the algorithm to output all valid configurations (or count the total number of valid solutions).

Welcome | QUEEN.C

QUEEN.C > N

```c
#include <stdio.h>
#define N 4
int board[N][N];
int col[N] = {0};
int diag1[2*N] = {0};
int diag2[2*N] = {0};

int solutionCount = 0;

void printBoard() {
    printf("Solution %d:\n", ++solutionCount);
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if (board[i][j] == 1)
                printf("V ");
            else if (board[i][j] == -1)
                printf("X ");
            else
                printf(". ");
        }
        printf("\n");
    }
    printf("\n");
}

void solve(int row) {
    if (row == N) {
        printBoard();
        return;
    }

    for (int c = 0; c < N; c++) {

        if (board[row][c] == -1)
            continue;

        if (col[c] || diag1[row + c] || diag2[row - c + N - 1])
            continue;

        board[row][c] = 1;
        col[c] = diag1[row + c] = diag2[row - c + N - 1] = 1;

        solve(row + 1);

        board[row][c] = 0;
        col[c] = diag1[row + c] = diag2[row - c + N - 1] = 0;
    }
}

int main() {

    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            board[i][j] = 0;

    // Blocked cells
    board[1][1] = -1;
    board[2][2] = -1;

    solve(0);

    if (solutionCount == 0)
        printf("No valid solutions\n");

    return 0;
}
```

```
PS C:\Users\Dimpal N\OneDrive\Desktop\clg study\C++ & DSA> cd "c
\QUEEN }
Solution 1:
. V . .
. X . V
V . X .
. . V .

Solution 2:
. . V .
V X . .
. . X V
. V . .

PS C:\Users\Dimpal N\OneDrive\Desktop\clg study\C++ & DSA>
```

---

3. Explain how you represent the board and efficiently track conflicts in rows, columns, and diagonals.

- <u>Board Representation</u>

  The board is represented using a 2D array of size N × N.

  - 0 represents an empty cell
  - 1 represents a vault placed in that cell
  - (-1) represents a blocked cell where a vault cannot be placed

  This representation helps in easily identifying valid and invalid positions for vault placement.

- ## Conflict Tracking

  To efficiently check whether placing a vault is safe, auxiliary arrays are used to track conflicts:

  1. Column Array:
     - Keeps track of columns that already contain a vault
     - If a column is occupied, another vault cannot be placed in that column
  2. Main Diagonal Array:
     - Represents diagonals from top-left to bottom-right
     - Index is calculated as (row + column)
  3. Anti-Diagonal Array:
     - Represents diagonals from top-right to bottom-left
     - Index is calculated as (row - column + N - 1)

- ## Efficiency

  Using these auxiliary arrays, conflict checking can be done in constant time O(1).

  This improves performance significantly compared to checking the entire board for conflicts each time, which would take O(N) time.

---

4. Analyze the worst-case time complexity of the backtracking solution in terms of N, and discuss how blocked cells affect the search space.

The worst-case time complexity of the N-Queens problem using backtracking is O(N!).

This is because for each row, there can be up to N possible positions, and the algorithm explores different combinations recursively.

However, the use of pruning reduces unnecessary computations. By checking column and diagonal conflicts before placing a vault, many invalid configurations are eliminated early.

Additionally, blocked cells reduce the search space, as fewer positions are available for placing vaults. This results in fewer recursive calls in practice, making the algorithm faster compared to the standard N-Queens problem without blocked cells.

---

5. Discuss techniques such as pruning invalid placements early, using auxiliary arrays for diagonals, and compare with a naive brute-force approach.

The solution uses several optimization techniques to improve efficiency:

1. Pruning:
   Invalid placements are eliminated early by checking column and diagonal conflicts before placing a vault. This avoids unnecessary recursive calls.

2. Auxiliary Arrays:

Arrays for columns and diagonals are used to check conflicts in constant time O(1), instead of scanning the entire board.

Comparison with Brute Force:

A naive brute-force approach would try all possible ways of placing vaults on the board, resulting in a time complexity of O(N^N), which is highly inefficient.

In contrast, the backtracking approach reduces the search space significantly by pruning invalid configurations early, making it much more efficient.